

Unidad III: Complejidad Computacional

Más NP-completitud (III)

Clase 20 - Lógica para Ciencia de la Computación/Teoría de la Computación

Prof. Miguel Romero

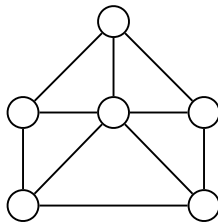
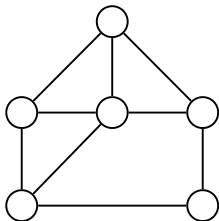
Definición:

Sea $G = (V, E)$ un grafo (no dirigido). Un k -coloreo de G es una función $c : V \rightarrow \{1, \dots, k\}$ tal que:

$$c(u) \neq c(v), \quad \text{para toda arista } \{u, v\} \in E.$$

Idea: c pinta cada nodo con uno de k colores de tal forma que los extremos de cada arista reciben colores distintos.

Ejemplo: ¿Cuáles de estos grafos tienen un 3-coloreo?



3-COL es NP-completo

Definamos el siguiente lenguaje:

$$3\text{-COL} = \{\langle G \rangle \mid G \text{ es un grafo que tiene un 3-coloreo}\}.$$

Teorema:

3-COL es NP-completo.

3-COL está en NP (¿por qué?). Basta ver que 3-COL es NP-hard.

3-SAT $\leq_p$ 3-COL

Veamos que 3-SAT $\leq_p$ 3-COL.

Sea $\varphi = C_1 \wedge \dots \wedge C_m$ una fórmula en 3-CNF con variables $\{x_1, \dots, x_n\}$.

Asumiremos que todas las cláusulas de φ tienen exactamente 3 literales.

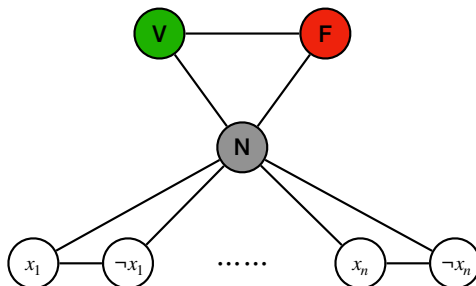
(¿por qué podemos asumir esto?)

La reducción computa un grafo G_φ definido como sigue:

- Hay 3 nodos especiales V, F y N. (verdadero, falso, neutro.)
- Por cada variable x_i de φ , hay dos nodos x_i y $\neg x_i$ en G_φ .

$3\text{-SAT} \leq_p 3\text{-COL}$

Los nodos se conectan de la siguiente forma:

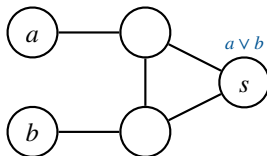


En cualquier 3-coloreo:

- V , F y N reciben 3 colores distintos.
- Llamaremos a estos colores **verdadero**, **falso** y **neutro**, respectivamente.
- Para cada $1 \leq i \leq n$:
Uno de los nodos en $\{x_i, \neg x_i\}$ recibe el color **verdadero** y otro el color **falso**.

3-SAT $\leq_p$ 3-COL

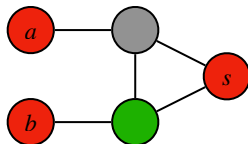
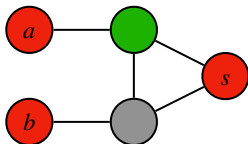
OR-gadget (para 2 entradas):



El nodo salida s simula la disyunción $a \vee b$:

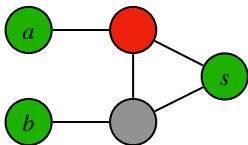
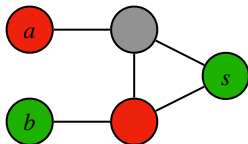
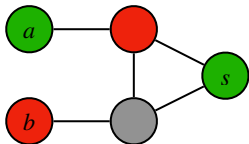
- Si a y b se pintan **falso**, **cualquier** 3-coloreo debe pintar s **falso**.
- Si a o b se pinta **verdadero**, **existe** un 3-coloreo que pinta s **verdadero**.

3-SAT $\leq_p$ 3-COL



Si a y b se pintan **falso**, **cualquier** 3-coloreo debe pintar s **falso**.

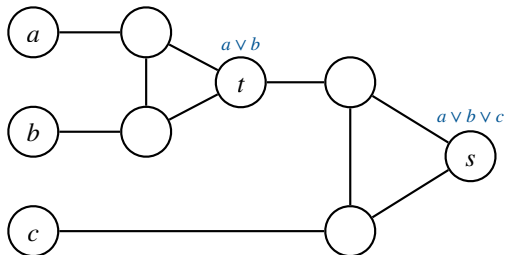
3-SAT $\leq_p$ 3-COL



Si a o b se pinta **verdadero**, **existe** un 3-coloreo que pinta s **verdadero**.

3-SAT $\leq_p$ 3-COL

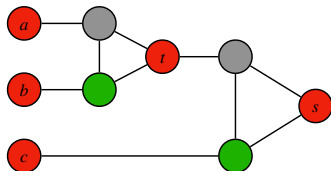
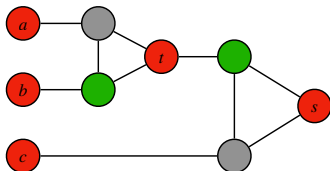
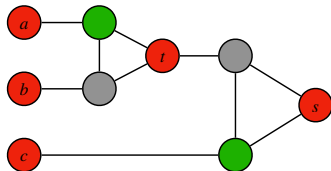
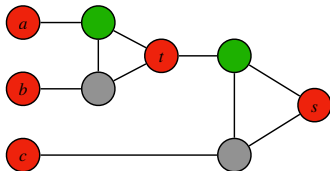
OR-gadget (para 3 entradas):



El nodo salida s simula la disyunción $a \vee b \vee c$:

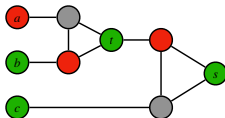
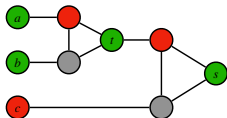
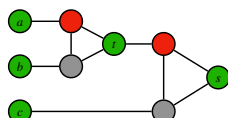
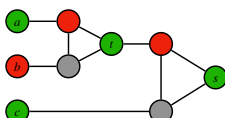
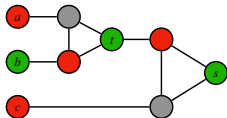
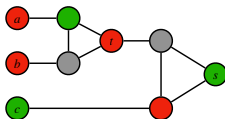
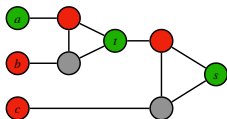
- Si a , b y c se pintan **falso**, **cualquier** 3-coloreo debe pintar s **falso**.
- Si a o b o c se pinta **verdadero**, **existe** un 3-coloreo que pinta s **verdadero**.

3-SAT $\leq_p$ 3-COL



Si a , b y c se pintan falso, **cualquier** 3-coloreo debe pintar s falso.

$3\text{-SAT} \leq_p 3\text{-COL}$



Si a o b o c se pinta **verdadero**, **existe** un 3-coloreo que pinta s **verdadero**.

3-SAT $\leq_p$ 3-COL

Sea $\varphi = C_1 \wedge \dots \wedge C_m$ una fórmula en 3-CNF con variables $\{x_1, \dots, x_n\}$.

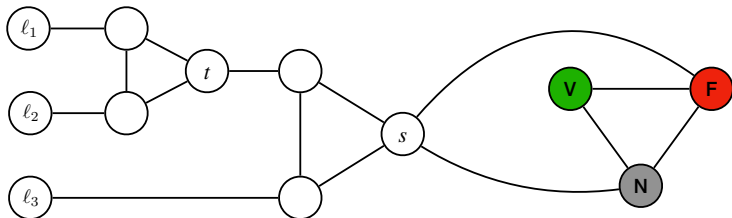
Asumiremos que todas las cláusulas de φ tienen exactamente 3 literales.

La reducción computa un grafo G_φ definido como sigue:

- Hay 3 nodos especiales V, F y N. (verdadero, falso, neutro.)
- Por cada variable x_i de φ , hay dos nodos x_i y $\neg x_i$ en G_φ .
- Por cada cláusula $C = (\ell_1 \vee \ell_2 \vee \ell_3)$ de φ , hay un OR-gadget.
(a , b y c del OR-gadget corresponden a los nodos ℓ_1 , ℓ_2 y ℓ_3 , resp.)
- Para cada uno de los OR-gadgets, agregamos aristas $\{s, F\}$ y $\{s, N\}$.

3-SAT $\leq_p$ 3-COL

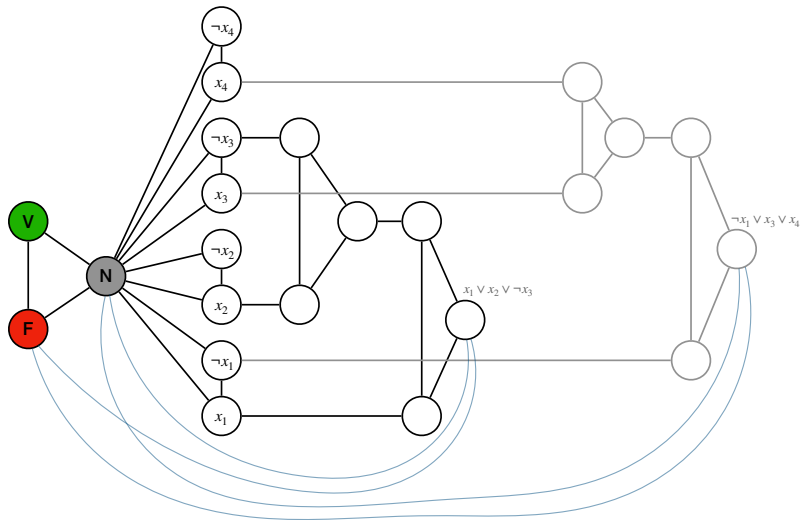
Por cada cláusula $C = (\ell_1 \vee \ell_2 \vee \ell_3)$ de φ , hay un OR-gadget:



- Si pintamos ℓ_1 , ℓ_2 y ℓ_3 **falso**, **no podemos** pintar el OR-gadget con 3 colores.
- Si pintamos ℓ_1 , ℓ_2 o ℓ_3 **verdadero**, **sí podemos** pintar el OR-gadget con 3 colores.

3-SAT $\leq_p$ 3-COL: ejemplo

Si $\varphi = (x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_3 \vee \neg x_4)$, entonces G_φ es:



3-SAT $\leq_p$ 3-COL: correctitud

La reducción se puede computar en tiempo polinomial.

Se cumple que:

$$\langle \varphi \rangle \in 3\text{-SAT} \iff \langle G_\varphi \rangle \in 3\text{-COL}.$$

($\Rightarrow$)

Si $\langle \varphi \rangle \in 3\text{-SAT}$, existe valuación σ que satisface a φ .

Podemos definir un 3-coloreo c como sigue:

- V, F, y N se pintan con 3 colores distintos. (verdadero, falso y neutro, resp.)
- Para cada literal ℓ , el nodo ℓ se pinta según el valor de verdad que le asigna σ . (nodo ℓ se pinta verdadero si $\sigma(\ell) = 1$; en caso contrario, se pinta falso.)
- Para cada cláusula $C = \ell_1 \vee \ell_2 \vee \ell_3$, al menos uno de los 3 nodos ℓ_1 , ℓ_2 o ℓ_3 queda pintado verdadero.
Luego, **podemos** pintar el OR-gadget asociado a C usando los 3 colores.
(el nodo de salida queda pintado verdadero.)

3-SAT $\leq_p$ 3-COL: correctitud

La reducción se puede computar en tiempo polinomial.

Se cumple que:

$$\langle \varphi \rangle \in 3\text{-SAT} \iff \langle G_\varphi \rangle \in 3\text{-COL}.$$

($\Leftarrow$)

Si $\langle G \rangle \in 3\text{-COL}$, existe 3-coloreo c para G_φ .

Podemos definir una valuación σ que satisface a φ como sigue:

- c pinta V, F, y N con 3 colores distintos. (verdadero, falso y neutro, resp.)
- Para cada $1 \leq i \leq n$: c pinta un nodo en $\{x_i, \neg x_i\}$ verdadero, y el otro falso.
- Para cada cláusula $C = \ell_1 \vee \ell_2 \vee \ell_3$: c pinta el OR-gadget asociado a C usando los 3 colores.

Luego, c debe pintar al menos uno de los 3 nodos ℓ_1 , ℓ_2 o ℓ_3 como verdadero.
(si los 3 nodos se pintan falso, el OR-gadget no se podría pintar.)

- Para cada literal ℓ , definimos $\sigma(\ell) = 1 \iff c$ pinta el nodo ℓ verdadero.

k -COL es NP-completo

Definamos el siguiente lenguaje, para $k \geq 2$:

$$k\text{-COL} = \{ \langle G \rangle \mid G \text{ es un grafo que tiene un } k\text{-coloreo} \}.$$

Teorema:

k -COL es NP-completo, para todo $k \geq 3$.

Demostración: Propuesta.

¿Qué pasa con 2-COL?